OptiTAC

A detailed screen-by-screen project demonstration

17 captured views | C/C++ Compiler Lab | 29 October 2026

Follow a program from editable source through tokens, syntax, types, intermediate instructions, guarded optimization and observed results. Then explore the machine-learning advisor, research audits, failure diagnosis and native GCC artifacts.

How this guide is organized

Each numbered screen has a detailed explanation followed by a readable screenshot excerpt. Excerpts are cropped from the screenshots captured during the browser walkthrough; the complete original images are embedded as PDF attachments so long panels remain available without shrinking an entire page of UI into unreadable text.

Demo anchor

The main example sums the integers from 1 through n. Supplied inputs 5, 7 and 10 produce printed sums 15, 28 and 55. Its selected optimization route reduces the displayed IR cost proxy from 30 to 24 units.

Reading the evidence correctly

A passing finite test suite is evidence for its checked cases. It is not a proof for every possible input. Static IR cost, interpreter steps, ML predictions and native syntax acceptance each answer different questions.

Prepared from the active application and the 17 screenshots saved in output/screen-demo. The screenshots preserve the actual narrow browser layout, including visible layout limitations.

Demo route and contents

01 Workspace and Tokens - pages 3-4

02 Syntax: the program tree - pages 5-6

03 Types: names, scopes and numeric rules - pages 7-8

04 Flow & IR: blocks and instructions - pages 9-10

05 Optimize / Autopsy: guarded search - pages 11-12

06 Results: compare observed behavior - pages 13-14

07 Counterfactual: omit one selected pass - pages 15-16

08 Structure: inspect the selected IR - pages 17-18

09 Logs: intermediate passes and rewrites - pages 19-20

10 ML advisor: predictions and measured evidence - pages 21-22

11 Validation-quality audit and witnesses - pages 23-24

12 Language scope and execution contract - pages 25-26

13 Research demo: discover and reduce a fault - pages 27-28

14 Native GCC: broader C++ artifacts - pages 29-30

15 Undefined behavior: signed overflow - pages 31-32

16 Learning controls, replay and export - pages 33-34

17 Experimental budgeted adaptive audit - pages 35-36

Suggested live presentation

Spend the first half on screens 1-6 to establish the compiler pipeline. Use screens 7-11 to explain transformation evidence and the role of ML. Finish with screens 12-17 to show scope, seeded fault detection, broader native syntax, explicit failures and learning interactions.

Workspace and Tokens

Start here to submit a program and see how source text becomes a stream of meaningful units. The same workspace stays available while you move through all six stages.

Controls and demo steps

Choose C11 or C++14, select a shipped example or edit Source code, and click Analyze program. Input cases & search settings lets you enter one stdin case per line, choose the interpreter budget and select a generated-input strategy. Reanalyze after changing the program or settings.

What this screen tells you

The initial example reads n, initializes total through constant expressions, sums 1 through n in a loop, prints the sum and returns it. Its cases are 5, 7 and 10. The Tokens screen reports 64 source tokens including directives, with operators separated from delimiters. Clicking a token selects its source span. The complete ledger provides a longer inspection view.

How to explain it

Explain that the lexer recognizes words and punctuation before the parser understands their relationships. Point out int as a keyword, total as an identifier, 2 as a numeric literal, * as an operator and ; as a delimiter. The native status reports an independent compile-only syntax check.

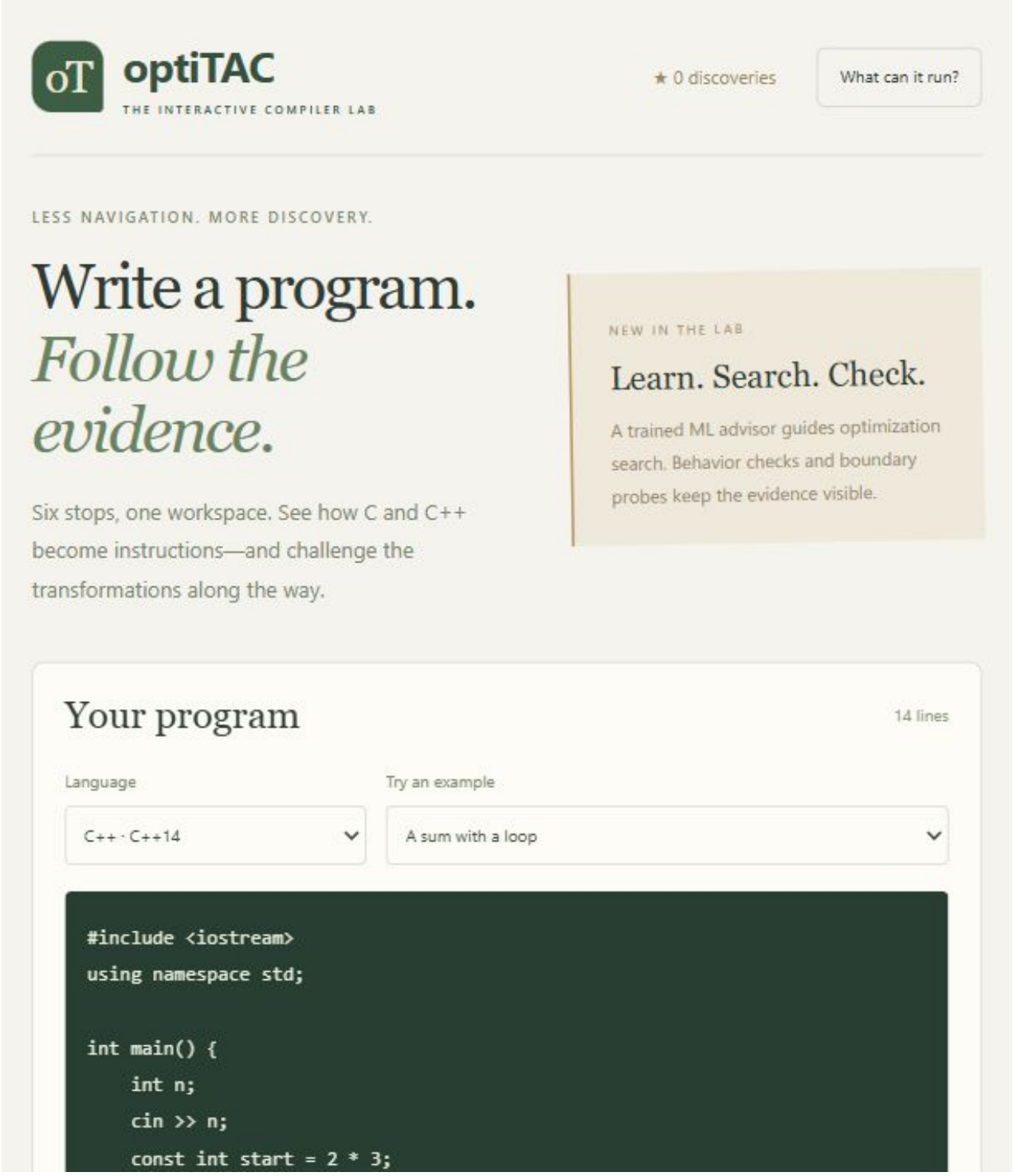
Interpretation and limits

Input settings are visible in the capture, but checkbox labels overlap at this narrow browser width. The PDF preserves that captured appearance; use the labels and keyboard focus if a pointer click targets an adjacent option.

SCREEN 01 | CAPTURED VIEW

Workspace and Tokens

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 01-tokens.jpg.



Syntax: the program tree

The Syntax stage turns the token stream into an abstract syntax tree (AST), representing the grammatical structure of the supported program.

Controls and demo steps

Select 02 Syntax and expand the tree nodes. Follow the main function into its body, then inspect declarations, assignments, expressions, the for loop, output and return statement. Expand only the subtree you want to discuss so the relationship between a parent node and its children stays clear.

What this screen tells you

A statement is represented by its role rather than just its printed spelling. For example, a binary expression has an operator and operand subtrees. The loop contains initialization, a condition, an update and a body. This structural representation is also the input to the reference interpreter used later in validation.

How to explain it

Use the loop condition i <= n to connect this screen back to Tokens. The lexer identifies i, <= and n separately; the parser groups them into a comparison expression. Then show where total += i sits inside the loop body.

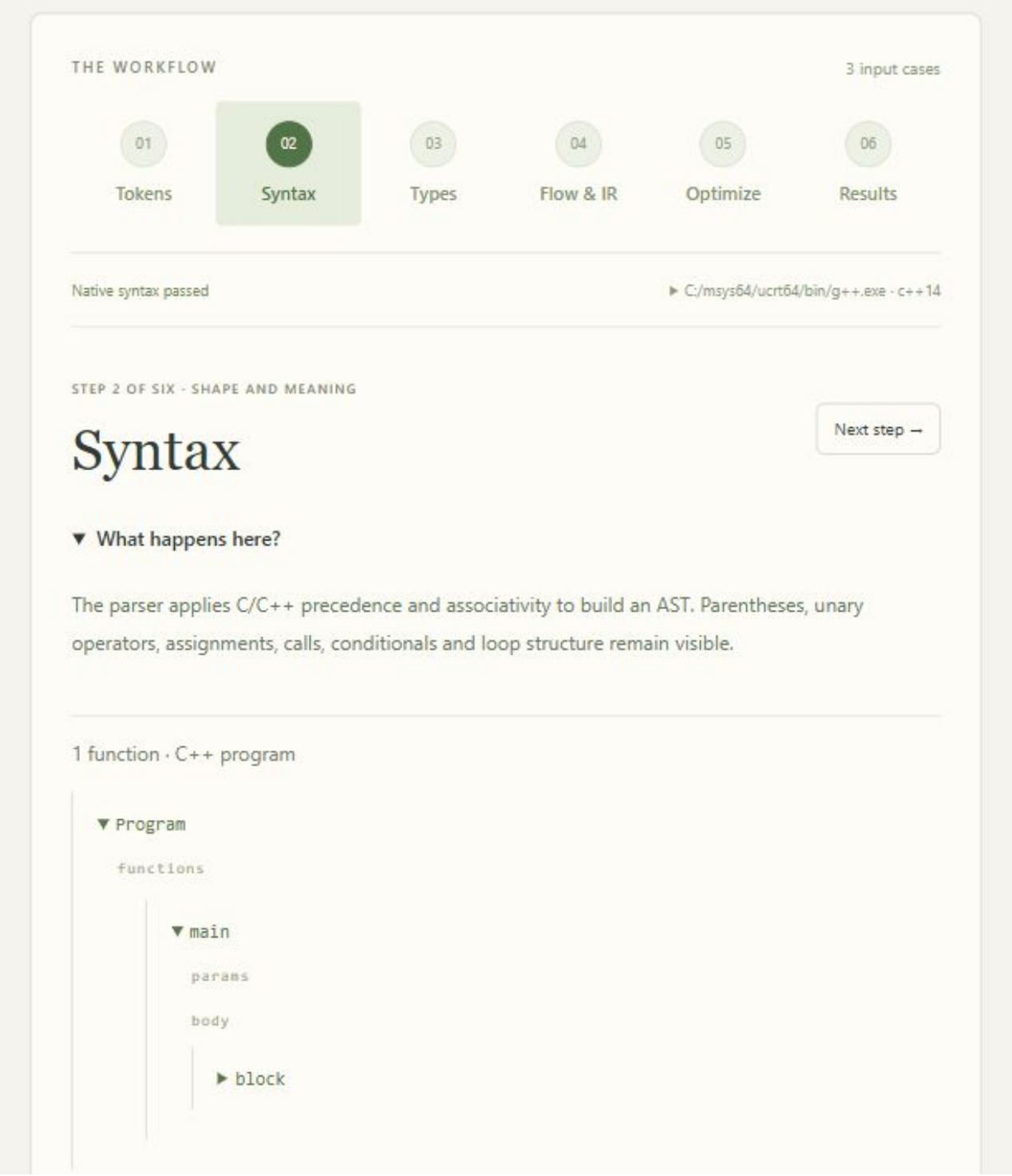
Interpretation and limits

The AST belongs to the documented visual subset. Native GCC acceptance of a broader C++ program does not guarantee that the visual parser or interpreter can represent and execute it.

SCREEN 02 | CAPTURED VIEW

Syntax: the program tree

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 02-syntax.jpg.



Types: names, scopes and numeric rules

The Types stage checks whether the parsed program makes semantic sense and records the types and storage rules of declarations.

Controls and demo steps

Select 03 Types. Read the table by Name, Type, Scope and Storage rule. Expand the numeric contract and semantic-rejection explanations to see which operations are supported and which conditions produce diagnostics.

What this screen tells you

In the loop example, n, start, total and i have integer types, while start is const. Declaration identity and lexical scope matter: two declarations with the same spelling in different blocks are different objects. The checker rejects undeclared names, duplicates in one scope, const writes, wrong function arity and invalid scalar operand combinations.

How to explain it

Explain that int uses checked signed 32-bit arithmetic and integer division truncates toward zero. float rounds to binary32 and double uses binary64. Checked casts require representable results. Overflow, division by zero and uninitialized reads produce diagnostics instead of invented values.

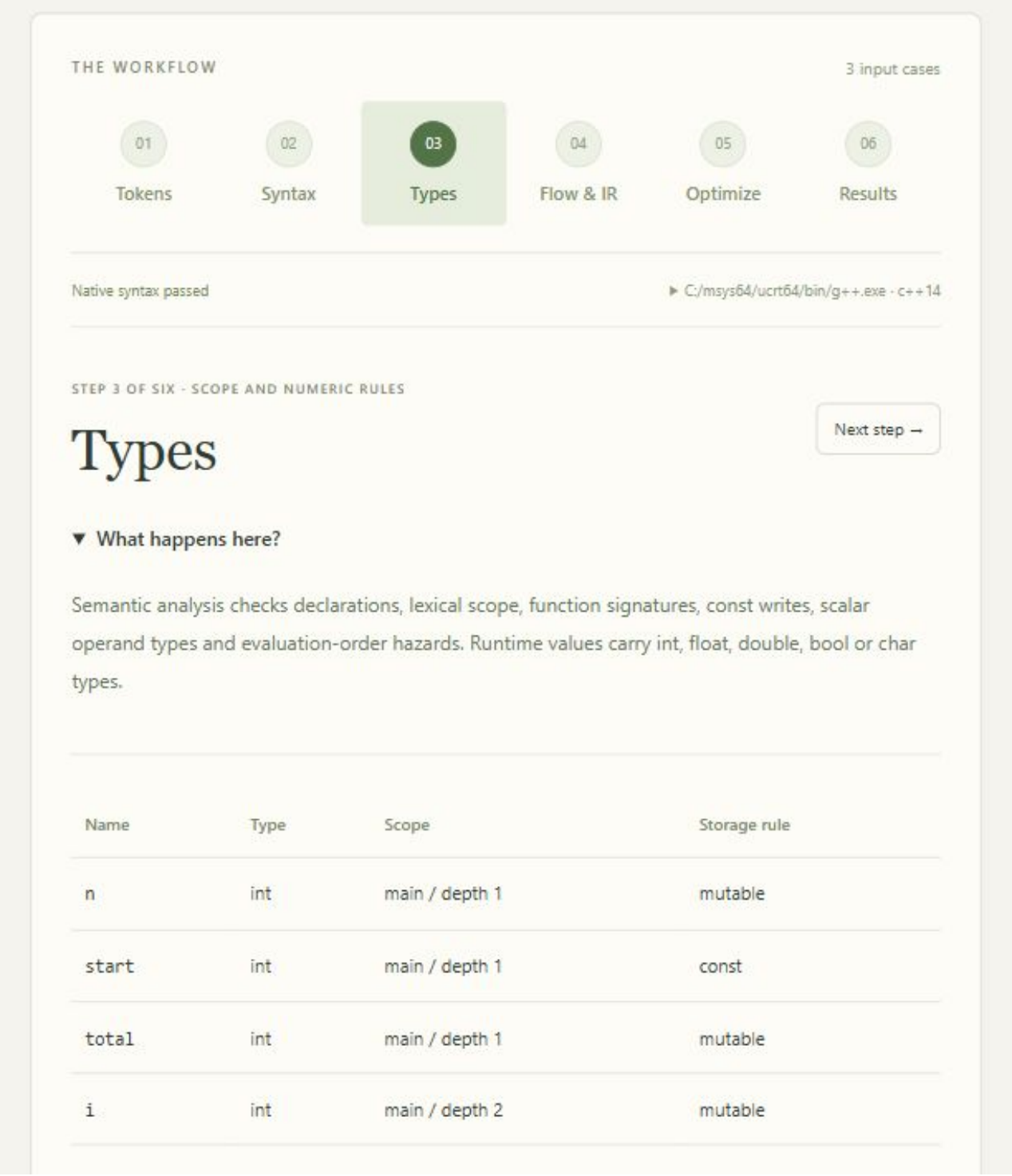
Interpretation and limits

A type check is not a guarantee that every execution is defined. A well-typed expression can still overflow or divide by zero for particular inputs; those conditions are observed during interpretation.

SCREEN 03 | CAPTURED VIEW

Types: names, scopes and numeric rules

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 03-types.jpg.



Flow & IR: blocks and instructions

This screen connects source-level control flow to typed intermediate representation (IR), the form that optimization passes transform.

Controls and demo steps

Select 04 Flow & IR. Click a basic-block card to inspect its instructions, use Next blocks links to follow successors, and switch to Show full original IR for the unoptimized instruction listing. Expand Backward liveness for this block and Reading the IR for the data-flow explanation.

What this screen tells you

The capture shows five blocks in the selected IR. Blocks group consecutive instructions; successor links describe possible transfers of control. Names such as total#3 identify a declaration, while $t names denote compiler temporaries. Scope and unwind instructions preserve block lifetimes through loops, break and continue.

How to explain it

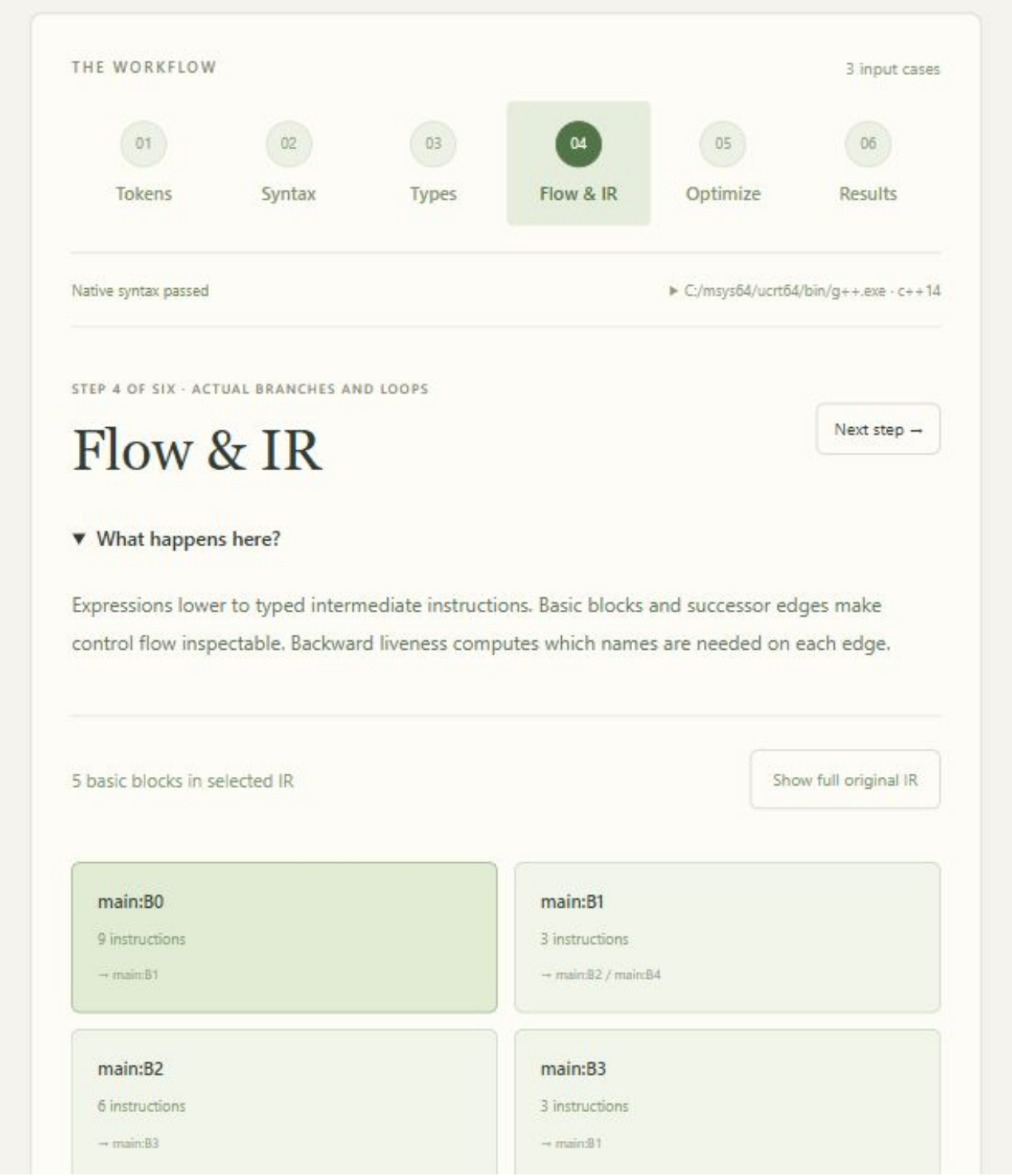
Walk from initialization into the loop condition, through the body and update, and back to the condition. Then explain liveness: IN = USE union (OUT minus DEF), while OUT combines successor IN sets. Iteration continues until these facts stop changing.

Interpretation and limits

The default block view describes selected IR, whereas the full-original toggle shows the original program. Interpreter instructions are educational IR and do not correspond one-to-one with processor instructions. Lazy expressions preserve short-circuit evaluation.

Flow & IR: blocks and instructions

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 04-flow-ir.jpg.



Optimize / Autopsy: guarded search

Optimization explores transformation sequences and keeps their behavioral evidence visible. Autopsy explains why a particular candidate was accepted or rejected.

Controls and demo steps

Select 05 Optimize and keep Autopsy active. Use Inspect a candidate or the numbered candidate buttons to choose a route. Read its verdict, route and cost, then compare reference and candidate stdout and typed returns for every supplied and eligible boundary input.

What this screen tells you

For the captured loop example, static cost changes from 30 to 24 units. Two transformed candidates are accepted, with Conditional constants followed by Effect-aware liveness selected. The displayed comparisons include three supplied cases and eight eligible generated probes. Input 5 prints sum=15 and returns integer 15 on both paths.

How to explain it

Explain the search as width-three, depth-four beam search, capped at 32 distinct transformed candidates. Conditional constants propagates facts over executable edges; reachability removes unreachable blocks; effect-aware liveness conservatively removes unused temporaries; jump cleanup simplifies transfers.

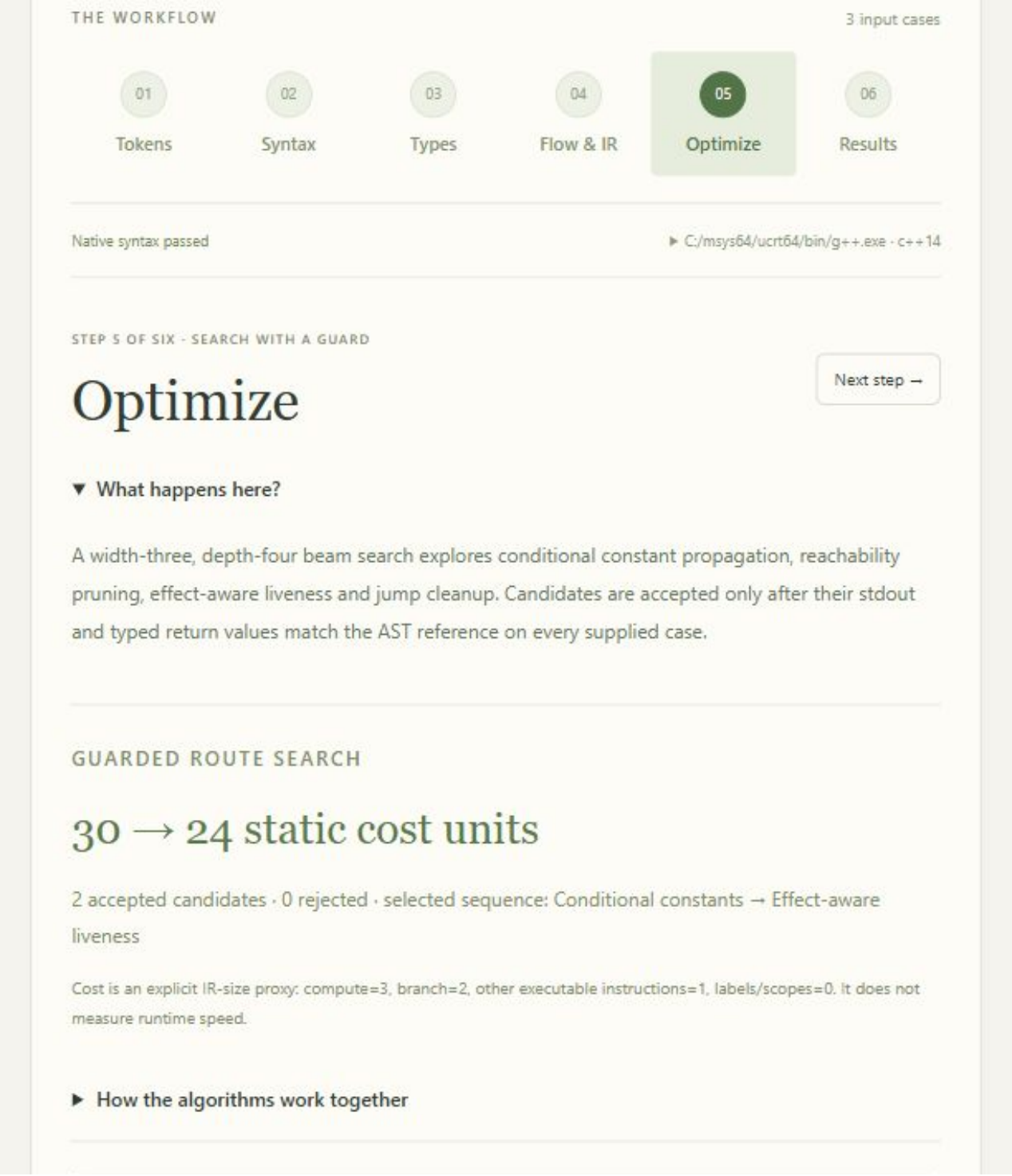
Interpretation and limits

The cost formula is an IR-size proxy: compute=3, branch=2, other executable instructions=1, labels/scopes=0. The 20% cost reduction is not a measured runtime speedup. Passing finite checks is evidence for those inputs, not a proof of equivalence.

SCREEN 05 | CAPTURED VIEW

Optimize / Autopsy: guarded search

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 05-optimize.jpg.



Results: compare observed behavior

Results presents the observable outcome of the reference AST, original IR and selected IR for each supplied input case.

Controls and demo steps

Select 06 Results and choose an Input case. Read the result banner, typed return and Program output, then compare the three execution rows. Expand Selected IR execution trace or AST reference execution trace to inspect individual recorded events.

What this screen tells you

The captured first case uses n=5. All three paths return 15 (int) and capture stdout as sum=15 followed by a newline. For the shipped cases 7 and 10, the expected sums are 28 and 55. Complete stdout and typed return values are compared, not just the headline number.

How to explain it

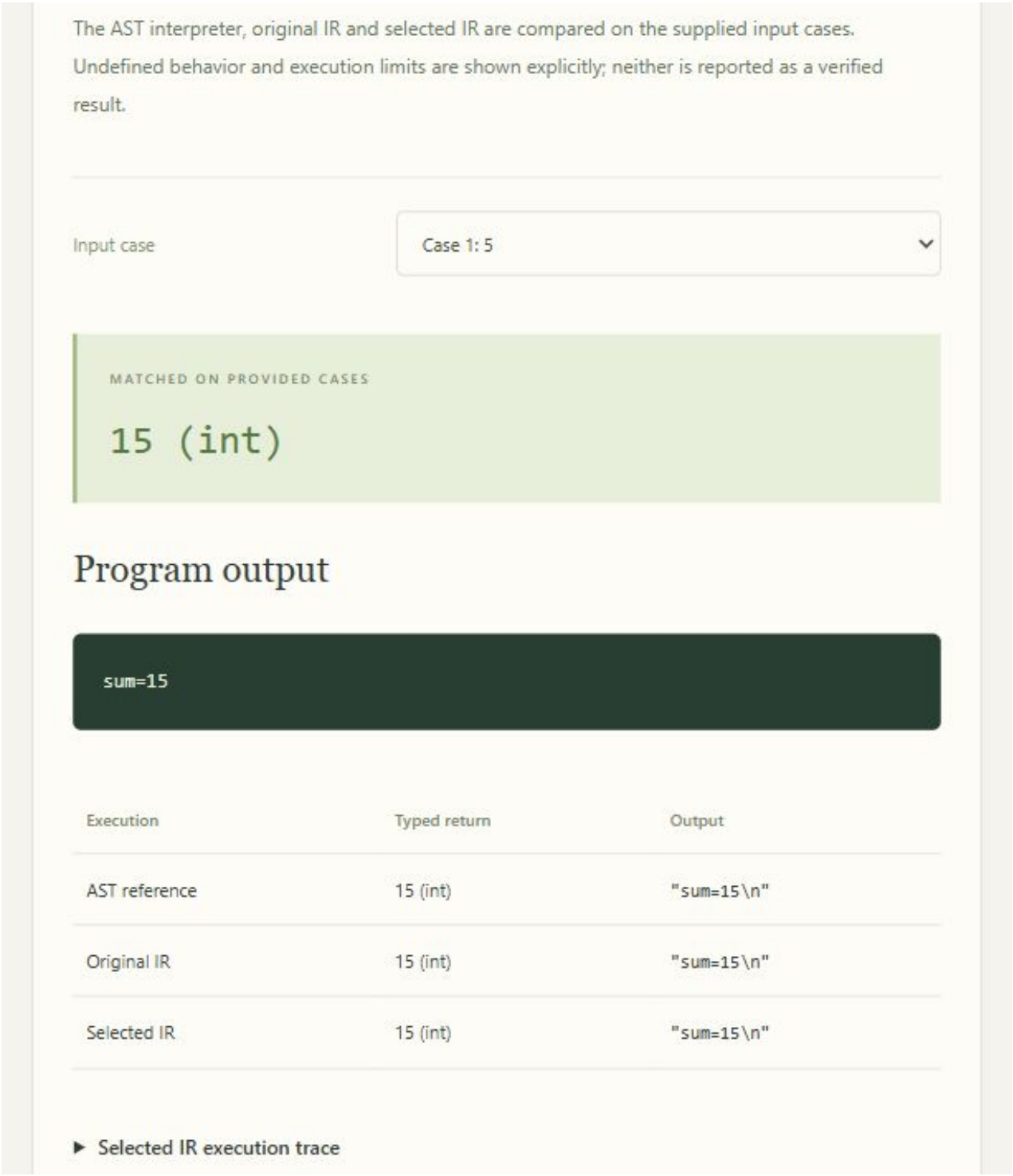
Explain that the program both prints a value and returns a value, which are separate observations. Trace events expose the instruction, function, source line, variable values and output so far. At most 160 events are retained; the interpreter step count is not CPU time.

Interpretation and limits

A matching case establishes agreement for that case. Undefined behavior and exhausted execution limits are not successful validation, even if two paths reach the same invalid or limited state. The AST and IR use separate control flow but share numeric primitives.

Results: compare observed behavior

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 06-results.jpg.



Counterfactual: omit one selected pass

This view helps explain how the selected sequence depends on each optimization pass by replaying the sequence with one pass omitted.

Controls and demo steps

Open Optimize, then Counterfactual. Read each row as an omission experiment: which pass was left out, the resulting cost, the remaining instruction count and whether observed behavior still matched on the checked cases.

What this screen tells you

The omission applies to the selected route. The remaining passes retain their order and are replayed. A higher resulting cost suggests the omitted pass helped this particular sequence reduce the IR proxy cost. A matching observation means the omission experiment preserved checked behavior.

How to explain it

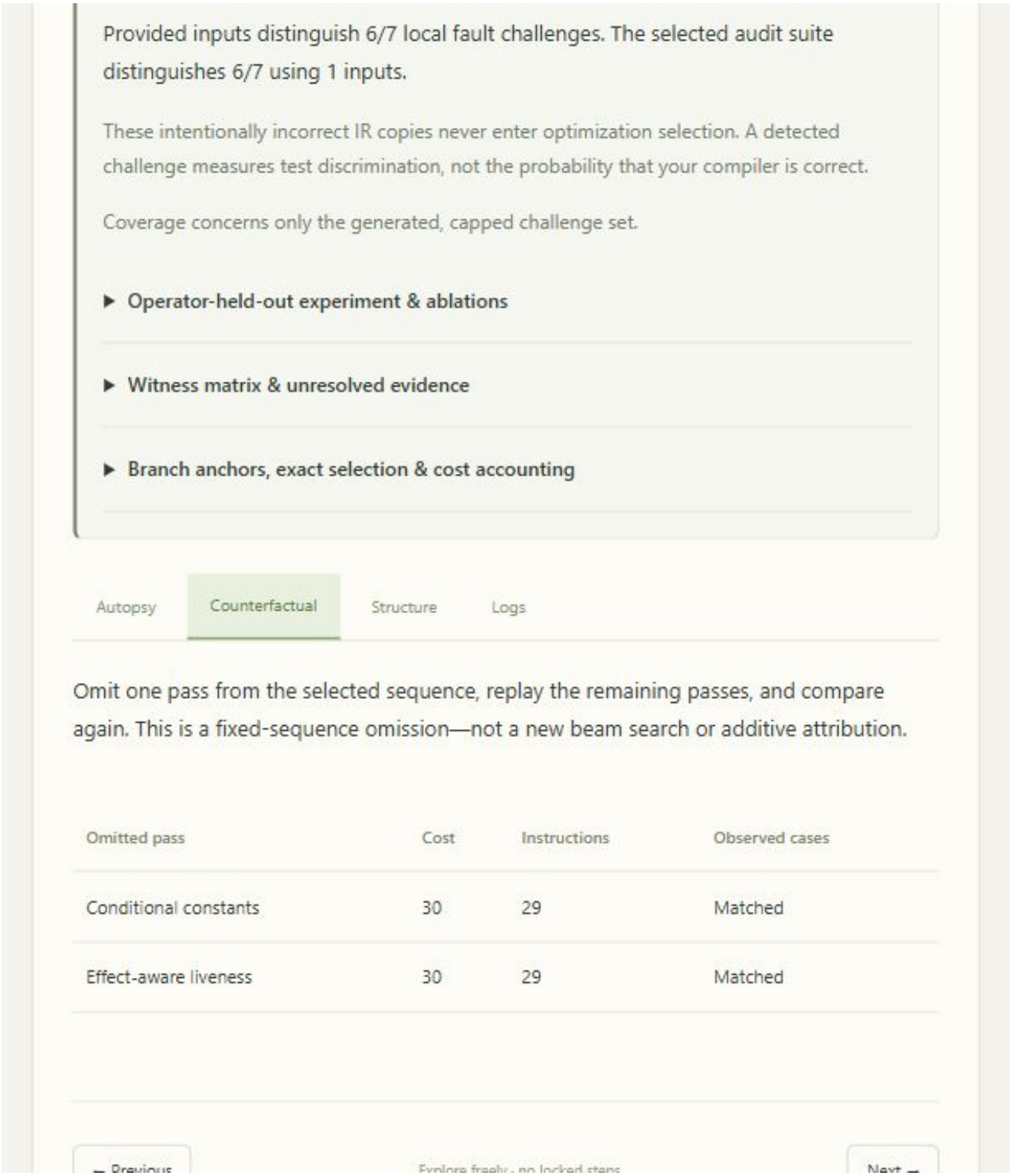
Compare the omission of Conditional constants with the omission of Effect-aware liveness. Discuss why discovering constant values can enable a later cleanup, so the effects of two passes can interact rather than add independently.

Interpretation and limits

This is a fixed-sequence omission experiment. It does not rerun beam search to find the best alternative route, prove causality in every program or allocate an additive share of savings to each pass. If no transformed route is selected, there may be no omission rows.

Counterfactual: omit one selected pass

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 07-counterfactual.jpg.



Structure: inspect the selected IR

Structure provides the complete selected optimized instruction listing so you can examine what the accepted transformation sequence actually produced.

Controls and demo steps

Open Optimize, then Structure. Read the instruction count and per-function IR listings. Compare with Flow & IR using Show full original IR to identify constants that were propagated or temporaries that were removed.

What this screen tells you

This view presents the selected optimized IR, rather than a source-code rewrite or machine-code disassembly. Instruction identifiers, typed operations and declaration-specific names help connect the result to the earlier block view and detailed rewrite logs.

How to explain it

Use the initialization in the loop example to explain how 2 * 3 becomes a known integer value and how the subsequent initialization of total can simplify. Then inspect which operations remain because the loop still depends on runtime input n.

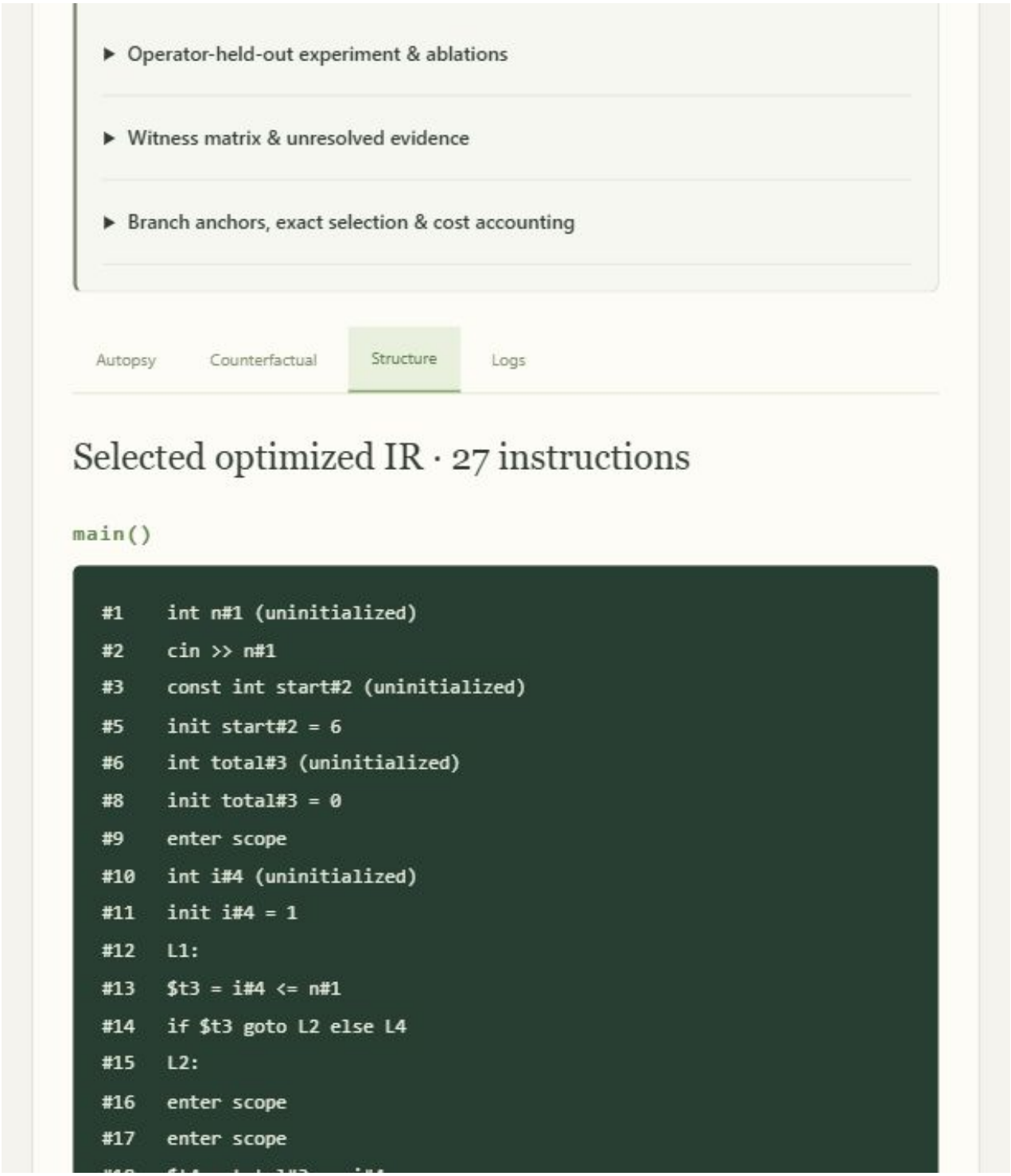
Interpretation and limits

A shorter listing alone does not establish correctness. Candidate acceptance depends on the guarded execution checks. Side-effecting and potentially trapping operations require conservative handling even when their values appear unused.

SCREEN 08 | CAPTURED VIEW

Structure: inspect the selected IR

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 08-structure.jpg.



Logs: intermediate passes and rewrites

Logs exposes the intermediate transformation process, connecting each recorded rewrite to a pass, instruction and explanation.

Controls and demo steps

Open Optimize, then Logs. Choose a candidate, select a Step in Intermediate process logs and read its worklist/liveness iterations, rewrite count and remaining instructions. Expand Detailed rewrite log, execution comparisons and Full IR at this step as needed.

What this screen tells you

Each rewrite record includes its function, instruction ID, before form, after form and reason. This makes it possible to separate a constant replacement from a later removal. Intermediate route prefixes are selectable when their corresponding candidate states are available.

How to explain it

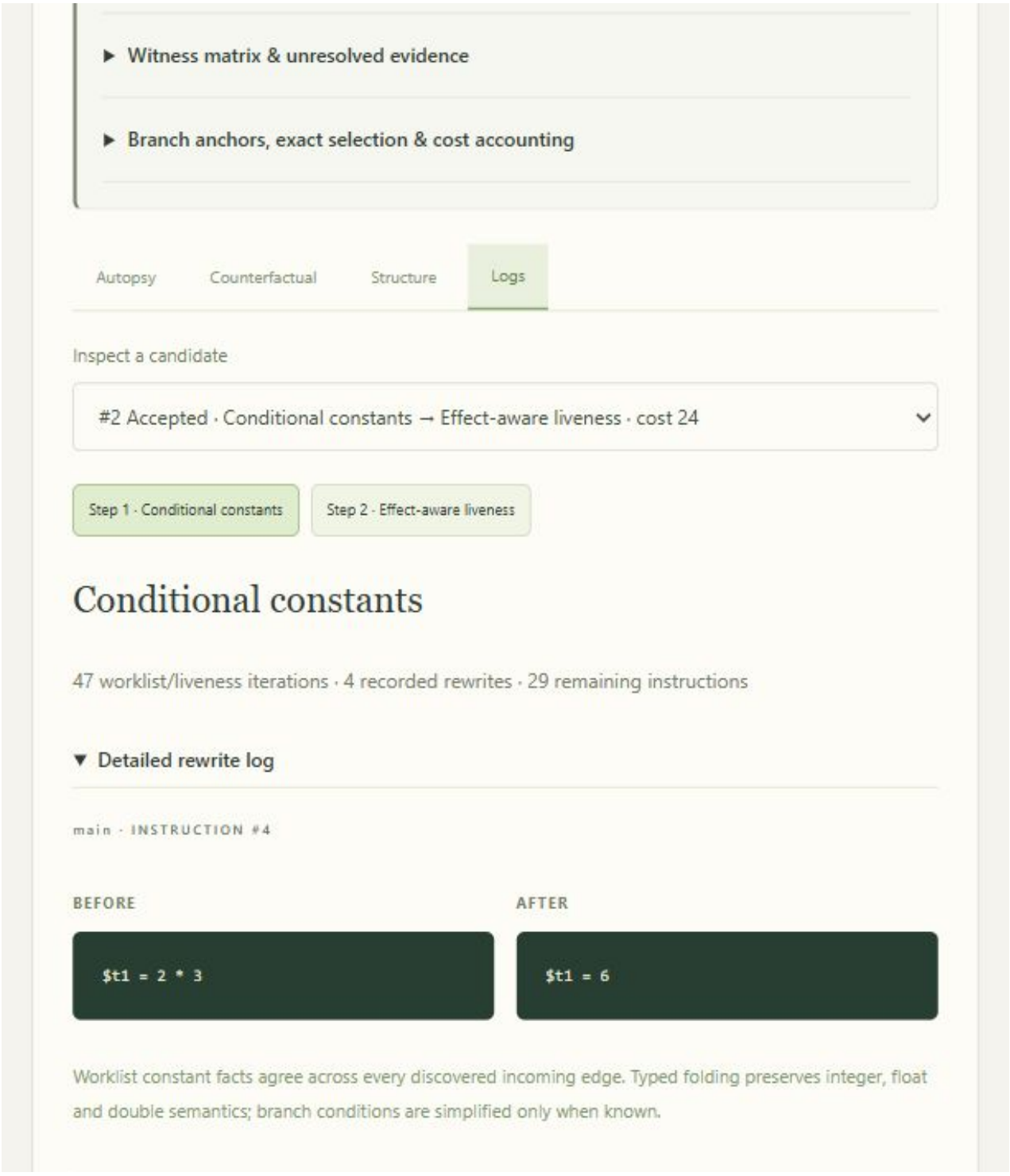
Demonstrate one change at a time: identify the original expression, read the rewritten instruction, and explain why the pass had enough information to act. Follow with the behavior comparisons for the same intermediate candidate.

Interpretation and limits

A rewrite log explains the implementation action; it is not a mathematical proof. A candidate with no recorded rewrites may show an explicit empty explanation. Deliberately broken experiments are identified separately and never selected as valid optimized output.

Logs: intermediate passes and rewrites

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 09-logs.jpg.



ML advisor: predictions and measured evidence

The bundled trained model guides which passes the search explores first. This panel distinguishes predicted benefit from measured cost and behavioral validation.

Controls and demo steps

Expand Predictions, evidence & why and Training & independent holdout results. Inspect predicted savings, calibration information, feature explanations and candidate-level comparisons between predictions and measured outcomes. Use Use trained ML guidance to run an ablation when comparing search behavior.

What this screen tells you

The active model is a bundled offline-trained random forest. Project documentation records 1,376 unique generated source programs and 4,784 measured IR states, partitioned by whole structural template. The panel exposes training and holdout context rather than presenting the prediction as a correctness certificate.

How to explain it

Describe the division of responsibilities: the forest ranks expansion order, the transformed IR determines measured cost, and the behavior guard determines acceptance. Outside the model training envelope, the advisor can fall back to default ordering.

Interpretation and limits

Predicted savings are estimates. They are not measured wall-clock speed, and a confident prediction cannot authorize an incorrect rewrite. Holdout results describe the stated experiment and dataset, not universal performance on arbitrary C/C++.

ML advisor: predictions and measured evidence

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 10-ml-advisor.jpg.



GUARDED ROUTE SEARCH

30 → 24 static cost units

2 accepted candidates · 0 rejected · selected sequence: Conditional constants → Effect-aware liveness

Cost is an explicit IR-size proxy: compute=3, branch=2, other executable instructions=1, labels/scopes=0. It does not measure runtime speed.

▶ How the algorithms work together

ML advisor + boundary audit

Trained model active

The trained forest ranks pass expansion order. Measured cost and behavior checks still decide acceptance.

A trained random forest predicts static cost reduction. It cannot authorize a rewrite or change the interpreter's semantics.

▼ Predictions, evidence & why

Pass on original IR	Predicted saving	Calibration error band
Reachability pruning	0.03	0.00 – 2.86
Conditional constants	0.03	0.00 – 0.11
Jump cleanup	0.00	0.00 – 0.50

Validation-quality audit and witnesses

This audit asks a different question from optimization acceptance: can the available tests expose intentionally incorrect local rewrites?

Controls and demo steps

Expand Witness matrix & unresolved evidence. Inspect which inputs distinguish which fault challenges, then review Branch anchors, exact selection & cost accounting and the operator-held-out experiment. Keep the distinction between supplied inputs, generated audit inputs and unresolved challenges visible.

What this screen tells you

For the captured loop example, supplied inputs distinguish six of seven local challenges. The selected audit suite also distinguishes six of seven using one input, leaving one unresolved challenge. The intentionally incorrect copies used for this audit never enter optimization selection.

How to explain it

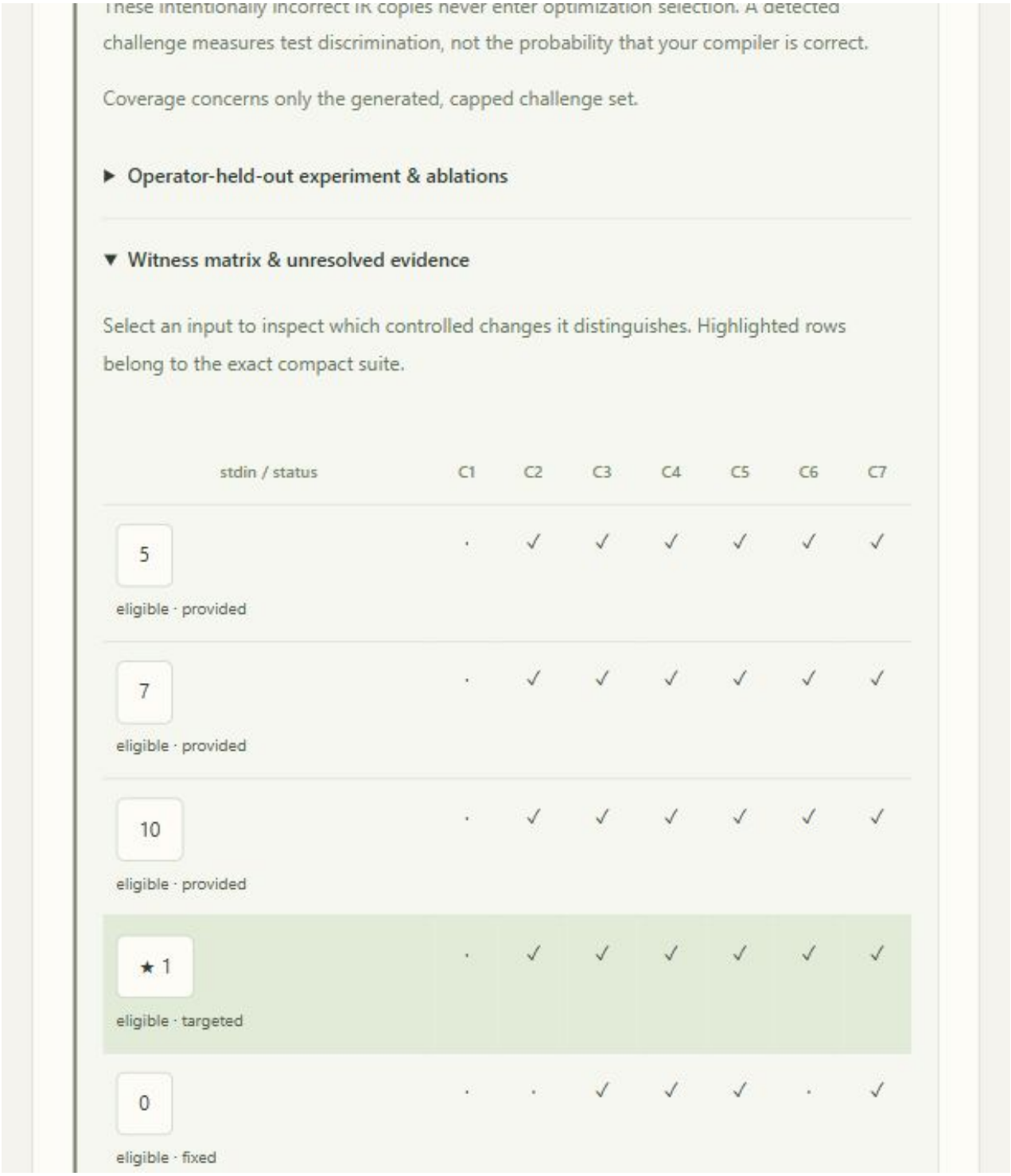
Explain a witness as an input whose reference and faulty execution disagree under an eligible, defined comparison. A compact portfolio attempts to retain useful witnesses while preserving relevant structural anchors, such as source branch outcomes and comparison-center proposals.

Interpretation and limits

The score measures discrimination against a generated, capped challenge set. It is not a probability that the compiler is correct. Unresolved challenges, finite budgets, planning overhead and the distinction between exact and greedy selection must remain explicit.

Validation-quality audit and witnesses

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 11-evidence-audit.jpg.



These intentionally incorrect IR copies never enter optimization selection. A detected challenge measures test discrimination, not the probability that your compiler is correct.

Coverage concerns only the generated, capped challenge set.

▶ **Operator-held-out experiment & ablations**

▼ **Witness matrix & unresolved evidence**

Select an input to inspect which controlled changes it distinguishes. Highlighted rows belong to the exact compact suite.

stdin / status	C1	C2	C3	C4	C5	C6	C7
5 eligible · provided	·	✓	✓	✓	✓	✓	✓
7 eligible · provided	·	✓	✓	✓	✓	✓	✓
10 eligible · provided	·	✓	✓	✓	✓	✓	✓
★ 1 eligible · targeted	·	✓	✓	✓	✓	✓	✓
0 eligible · fixed	·	·	✓	✓	✓	·	✓

Language scope and execution contract

What can it run? documents the boundary between supported visual execution and the broader syntax accepted by the local native compiler.

Controls and demo steps

Click What can it run? in the header. Read Visual execution, Native syntax checking and Explicit boundaries. Use this panel before choosing a demo that depends on a feature such as classes or containers.

What this screen tells you

The visual path supports scalar declarations, assignments, casts, unary/binary operators, branches, loops, break/continue, lexical scope, scalar functions and bounded recursion. It includes basic printf/scanf, cout/cin and numeric math helpers. The native service checks supported broader C11/C++14 translation units.

How to explain it

Contrast the loop example with the broader class/vector example. The loop can pass through the full visual pipeline. The broader example can be native-syntax-valid while visual execution is unavailable, in which case the site exposes that boundary and offers native artifacts.

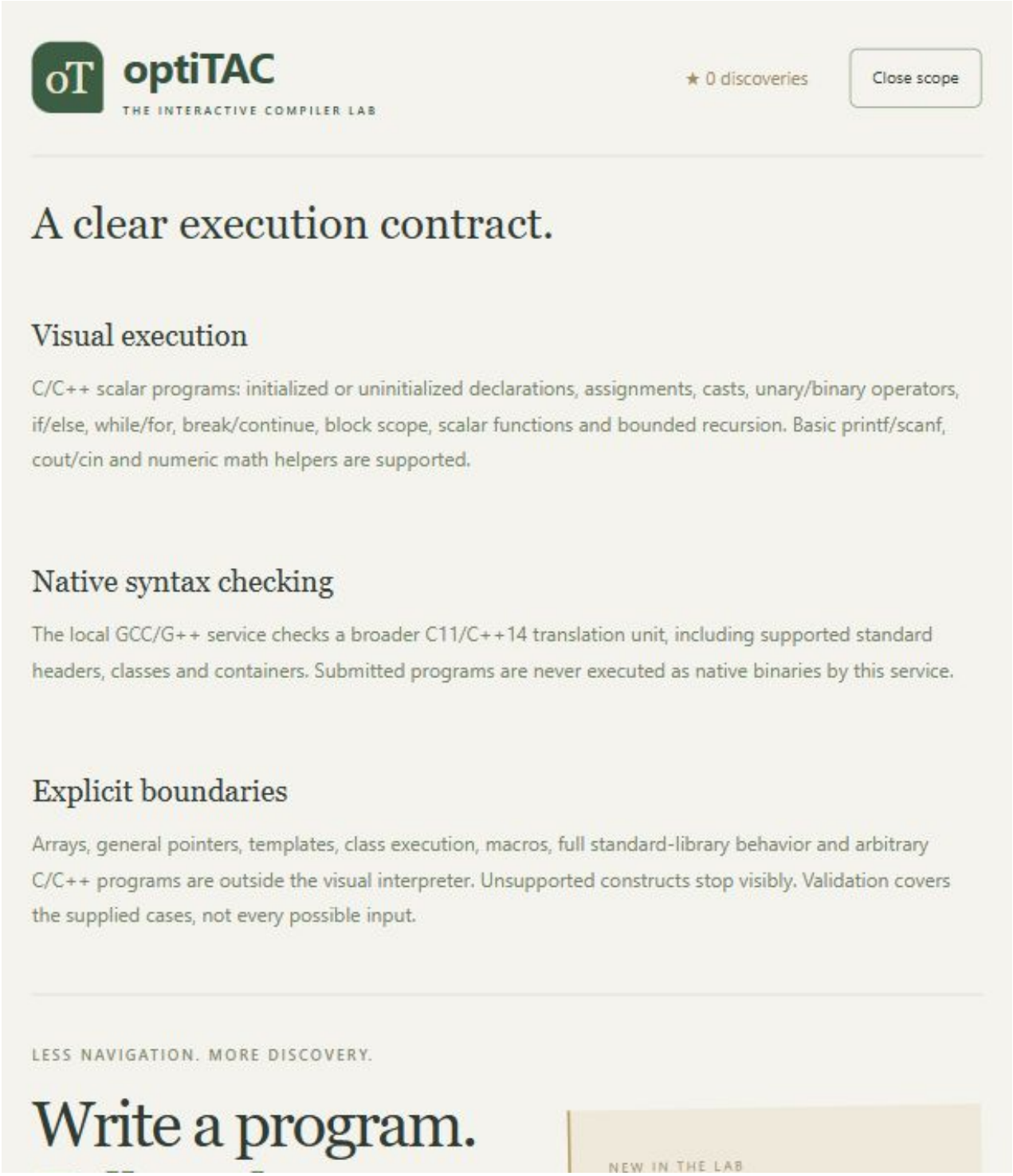
Interpretation and limits

General pointers, arrays, templates, class execution, macros, full standard-library behavior and arbitrary C/C++ are outside the visual interpreter. Submitted programs are never run as native binaries by the service. Its compile-only restrictions do not constitute a security sandbox.

SCREEN 12 | CAPTURED VIEW

Language scope and execution contract

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 12-language-scope.jpg.



Research demo: discover and reduce a fault

The research demo demonstrates why supplied examples alone can miss a localized mistake and how targeted generation can produce a useful counterexample.

Controls and demo steps

Select Research demo: find the hidden fault and analyze. Open Optimize / Autopsy and choose the rejected Broken candidate. Read the supplied versus generated checks and the Automatic failure diagnosis table, including before/after stdin and ordered pass actions.

What this screen tells you

The source reads n and an unused second integer. It returns 7 when n equals 137 and 2 otherwise. The supplied input 300 999 misses that branch. Comparison-targeted probing can exercise n=137 and expose the deliberately introduced localized fault; the documented reduced witness is 137 0.

How to explain it

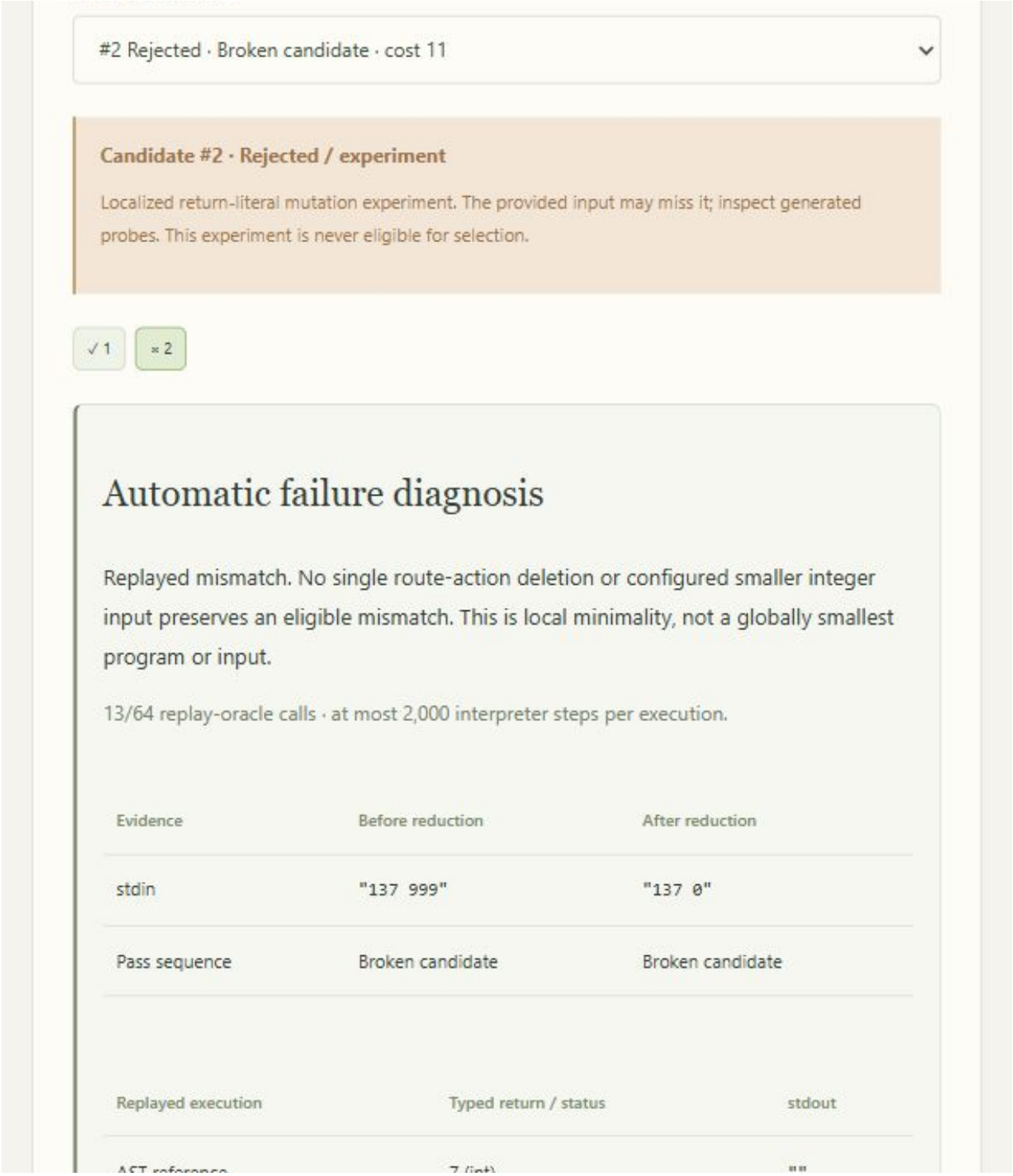
Explain that reducing the unused input value makes the failing example easier to understand while retaining the mismatch. The diagnosis replays the AST reference, original IR and reduced candidate under bounded oracle calls. Reduction applies to stdin values and ordered pass actions, not source statements.

Interpretation and limits

The fault is deliberately seeded for demonstration and is always ineligible for selection. Only defined reference executions agreeing with original IR qualify for the diagnosis. Timeouts cannot establish a counterexample, and bounded local reduction does not imply global minimality.

Research demo: discover and reduce a fault

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 13-hidden-fault.jpg.



Native GCC: broader C++ artifacts

Native artifacts extend the lab with the compiler own representations when a valid C/C++ program exceeds the visual interpreter subset.

Controls and demo steps

Choose Broader C++ syntax check, analyze, then open Flow & IR. The alert explains why visual execution is unavailable. Click Generate GCC IR & CFG and expand the available original tree, pre-optimization GIMPLE, CFG and optimized GIMPLE sections.

What this screen tells you

The example declares a Scores class holding std::vector<int>, assigns values, sorts them and returns the first element. Native syntax checking accepts the supported translation unit, while the visual path stops at the unsupported class construct. Generated dumps provide real GCC evidence without claiming visual execution.

How to explain it

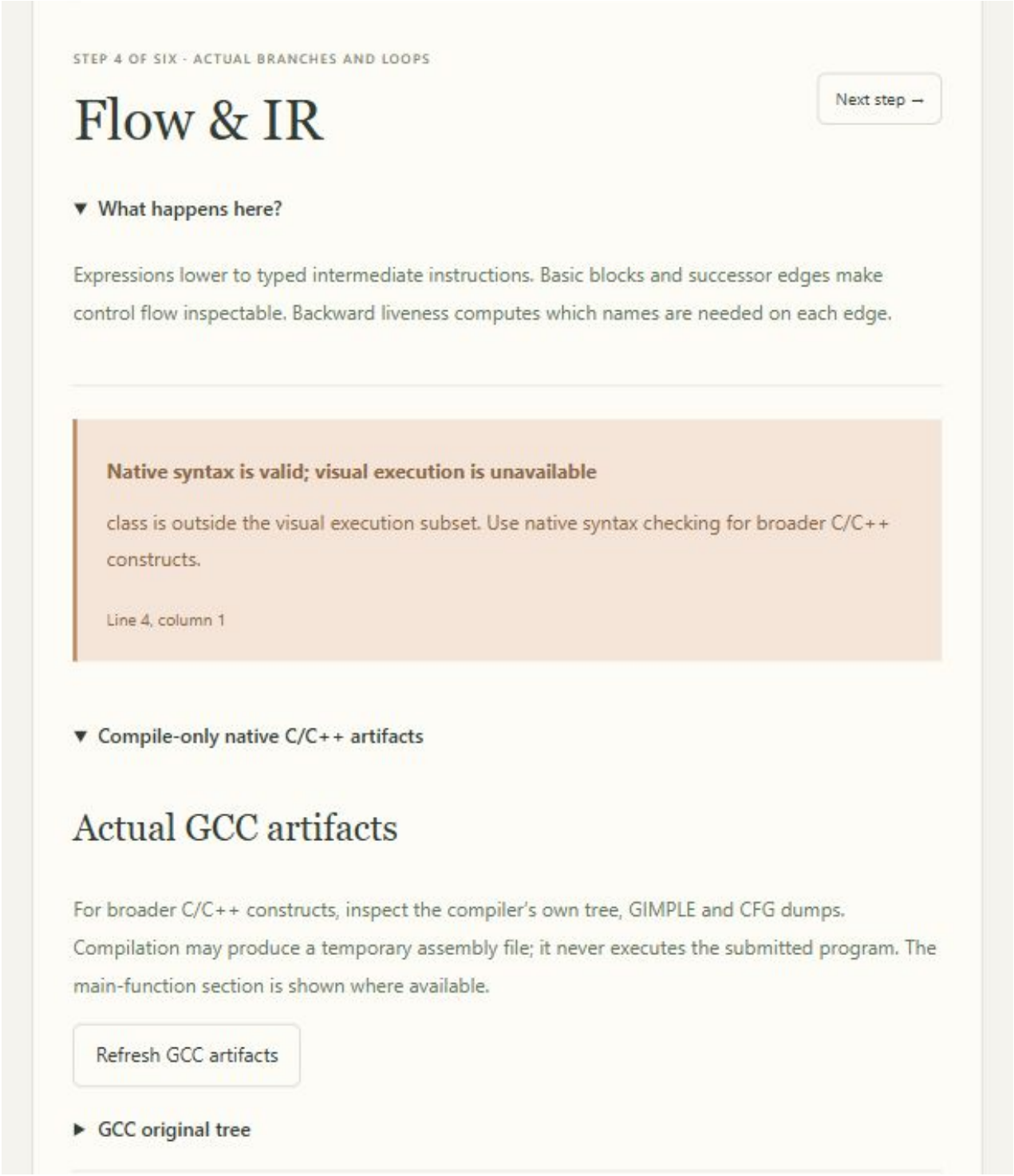
Use the main-function section where available, rather than reading every library-generated helper. Explain GIMPLE as a compiler intermediate form and compare the pre-optimization and -O1 views. The screenshot is an overview with the very long GIMPLE section collapsed after generation.

Interpretation and limits

Compilation can create temporary assembly and compiler dumps, but the submitted program is not executed as a native binary. GCC artifacts do not establish equivalence with the visual interpreter and may include substantial standard-library detail.

Native GCC: broader C++ artifacts

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 14-native-artifacts.jpg.



Undefined behavior: signed overflow

The overflow example demonstrates how the lab handles an execution for which the documented signed-integer semantics do not provide a valid result.

Controls and demo steps

Select Catch signed overflow, analyze and open Results. Read the UNDEFINED BEHAVIOR DETECTED banner and the diagnostic details, then inspect any Program output before the diagnostic and available execution traces.

What this screen tells you

The interpreter uses checked signed 32-bit integers. When arithmetic exceeds the representable range, the site reports a diagnostic instead of silently wrapping and presenting the wrapped value as a valid C/C++ result. Output already captured before the failure is labeled partial.

How to explain it

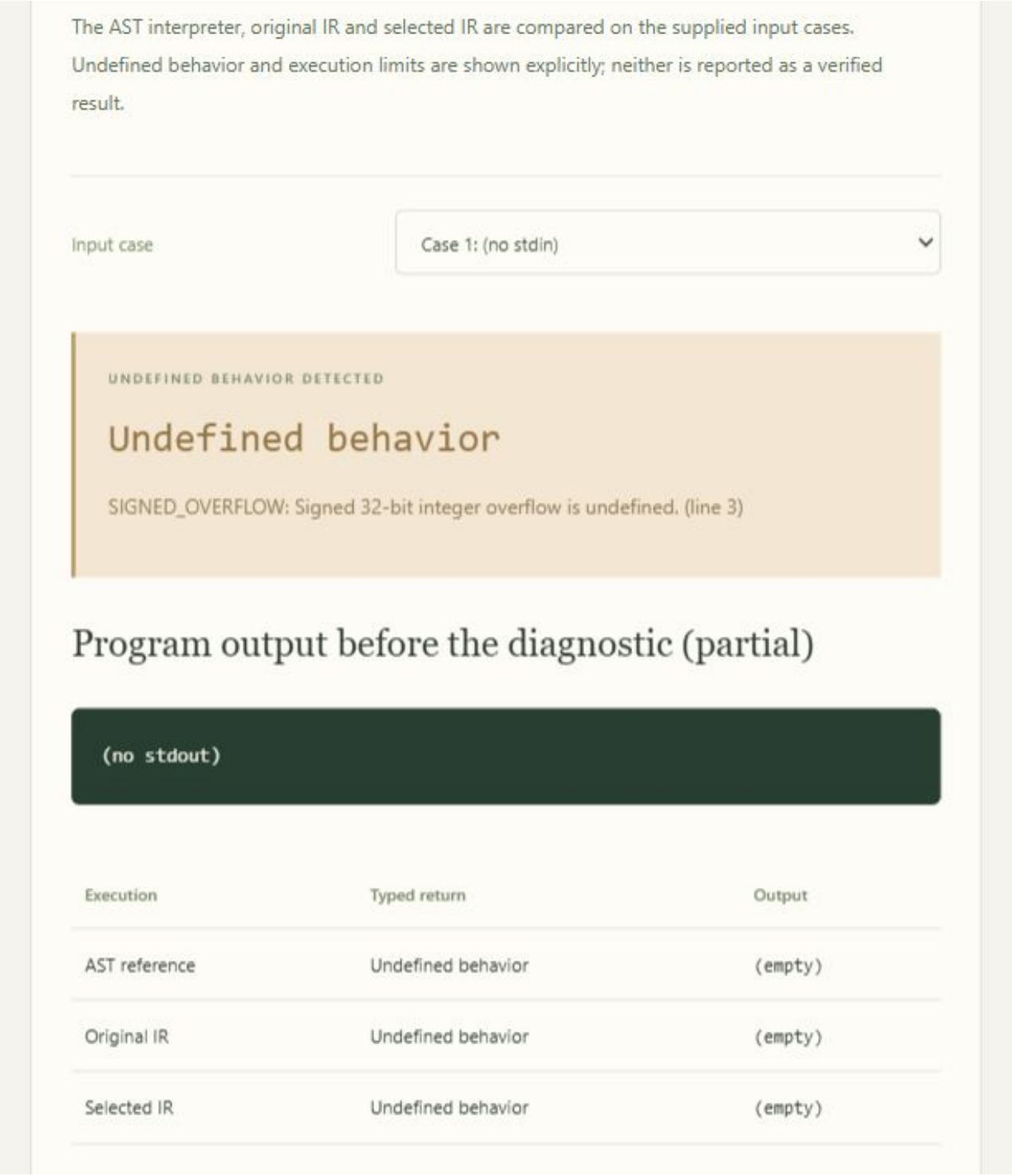
Contrast this screen with the successful sum example. A normal result can compare stdout and a typed return. An undefined execution cannot be certified simply because two internal paths encounter the same failure.

Interpretation and limits

Native syntax acceptance does not remove runtime undefined behavior. Changing the interpreter budget cannot make signed overflow valid. For inconclusive step-limit cases, the correct interpretation is similarly that the evidence did not establish a successful matched execution.

Undefined behavior: signed overflow

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 15-undefined-behavior.jpg.



Learning controls, replay and export

The learning controls encourage prediction and exploration while preserving direct access to compiler evidence.

Controls and demo steps

For the sum example, expand Quick challenge and enter sum=15 for the first case. Click Check prediction. The captured response confirms the prediction and awards a discovery star. Play walkthrough automatically advances through the six stages; Pause walkthrough stops the replay.

What this screen tells you

The challenge asks about printed output, so the answer omits the trailing newline rather than using only the return value 15. Discovery stars persist locally. Every stage remains freely accessible; the learner is not required to complete a challenge before inspecting later evidence.

How to explain it

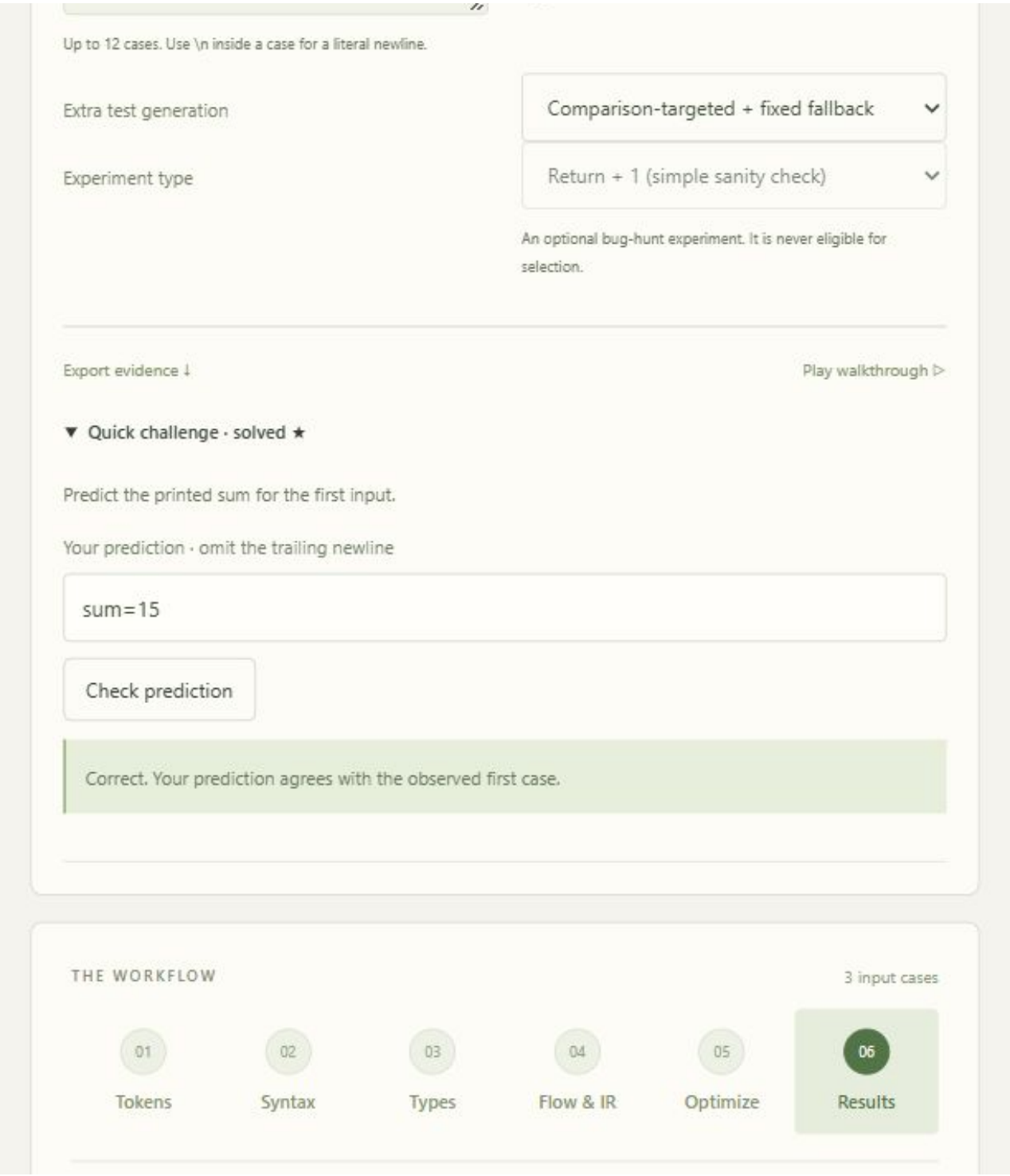
Use the sequence predict, run, compare and explain. After solving the first case, change the selected case to 7 or 10 and reason through the different sum. Export evidence saves a JSON record for offline inspection of the analysis, checks and research evidence.

Interpretation and limits

A solved challenge is a learning interaction, not an additional proof of program correctness. The screenshot records the successful challenge. Replay was exercised during capture; export is an available application control rather than an additional generated deliverable in this PDF.

Learning controls, replay and export

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 16-challenge.jpg.



Experimental budgeted adaptive audit

Adaptive auditing explores how to allocate a limited validation budget while retaining an inspectable record of scheduling decisions.

Controls and demo steps

Enable Budgeted adaptive audit (experimental), reanalyze and open Optimize. Inspect the adaptive evidence trail and validation-quality panel. Compare with a run where the option is disabled while holding source, supplied inputs and other search settings constant.

What this screen tells you

The capture confirms the experimental option is active and that an Adaptive evidence trail is produced. The recorded scheduling evidence belongs to the validation audit. It should be interpreted alongside witness coverage, unresolved challenges and execution cost rather than only the headline optimization savings.

How to explain it

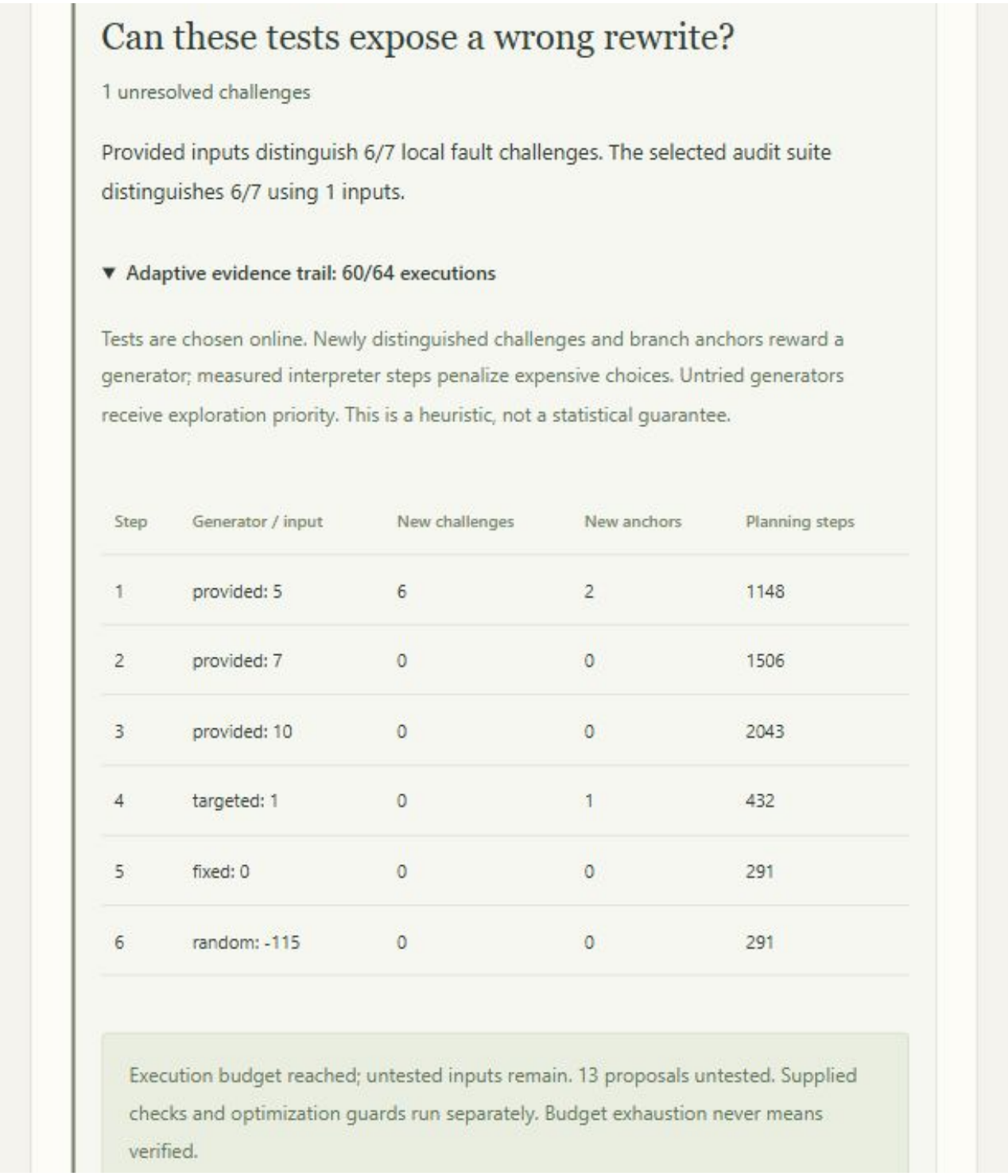
Explain the motivation as a constrained testing problem: under a budget, choose what to check next and retain the evidence needed to explain that choice. The scheduling path can change which useful distinctions are discovered within the available resources.

Interpretation and limits

This feature is explicitly experimental. It does not authorize incorrect optimization or replace defined behavior checks. The narrow captured layout caused a pointer checkbox interaction to miss; keyboard selection was used to activate the intended option before this screenshot was taken.

Experimental budgeted adaptive audit

Focused excerpt from the actual captured screen. The complete uncropped screenshot is attached to this PDF as 17-adaptive-audit.jpg.



Can these tests expose a wrong rewrite?

1 unresolved challenges

Provided inputs distinguish 6/7 local fault challenges. The selected audit suite distinguishes 6/7 using 1 inputs.

▼ **Adaptive evidence trail: 60/64 executions**

Tests are chosen online. Newly distinguished challenges and branch anchors reward a generator; measured interpreter steps penalize expensive choices. Untried generators receive exploration priority. This is a heuristic, not a statistical guarantee.

Step	Generator / input	New challenges	New anchors	Planning steps
1	provided: 5	6	2	1148
2	provided: 7	0	0	1506
3	provided: 10	0	0	2043
4	targeted: 1	0	1	432
5	fixed: 0	0	0	291
6	random: -115	0	0	291

Execution budget reached; untested inputs remain. 13 proposals untested. Supplied checks and optimization guards run separately. Budget exhaustion never means verified.

A complete demo script

1. Establish the input

Load A sum with a loop. Explain that n is supplied through stdin. Keep cases 5, 7 and 10. Click Analyze program and wait for native syntax checking to finish.

2. Follow the compiler pipeline

Show Tokens, expand a Syntax node, read the Types table and follow a basic block in Flow & IR. Connect each representation back to the same loop in the source.

3. Explain optimization evidence

Show 30 to 24 static cost units. Inspect the selected candidate, a matched case, the counterfactual omissions and one rewrite log. Explain why the cost proxy does not measure runtime speed.

4. Separate predictions from decisions

Open the ML explanations. Describe the model as a search-order advisor. Then open the witness matrix and distinguish candidate acceptance from test-suite discrimination.

5. Demonstrate a failure deliberately

Load the hidden-fault example and choose the rejected Broken candidate. Explain why the supplied case misses n=137 and how targeted probing and reduction make the fault observable.

6. Close with boundaries and learning

Show the native class/vector example, the overflow diagnostic and the language contract. Return to the sum example, solve the prediction challenge and show replay and evidence export.

Reference notes and attachments

Project sources used

README.md: active project overview, run instructions, validation scope and model dataset.

src/lab/Views.tsx: types, flow/IR, optimization tabs, logs and results behavior.

src/lab/FailureView.tsx: failure reproduction and bounded reduction semantics.

output/screen-demo/*.jpg: 17 screenshots from the browser walkthrough.

Screenshot attachments

All 17 original screenshots are embedded as attachments. A PDF reader with an Attachments panel can open or save them at full image resolution. The focused excerpts in the guide are reading aids; they do not replace the original captures.

Practical limitations of this capture

The browser capture was 612 pixels wide, producing long responsive pages. Some settings labels overlap in that layout. Long expanded audit and ML panels repeat the shared workspace above their content. To preserve readability, this guide uses focused excerpts and retains the complete originals as attachments.

Local demonstration server

The walkthrough used http://127.0.0.1:5187/. This address works only while the project development server is running on the same computer. The PDF and its attached screenshots can be read offline.